Coding Assignment 3
Backpropagation and Optimization

Overview

This activity walks you through building, training, and evaluating a neural network for regression using the **Family Income and Expenditure Survey** dataset from the Philippine Statistics Authority. You will compare **13 gradient-based optimizers** implemented in Keras, interpret their convergence behavior, and connect your observations to the mathematics of backpropagation.

Dataset: Family Income and Expenditure Survey, Philippine Statistics Authority.
Source: Family Income and Expenditure Survey
Prediction target: Total Food Expenditure (continuous regression)

Warning: Complete all coding tasks in your own Jupyter notebook, following the instructions given in each part of this sheet. Answer the written questions *in this sheet* based on your observations and course concepts. **Do not copy code outputs verbatim as answers.** Explain using your own words and equations.

Learning Objectives

By the end of this activity, you should be able to:

1. Trace the forward and backward pass of backpropagation using the chain rule.
2. Distinguish adaptive from non-adaptive gradient-descent optimizers and articulate their trade-offs.
3. Interpret training and validation loss curves to diagnose convergence, underfitting, and overfitting.
4. Compare the Adam optimizer family mathematically and empirically.
5. Explain vanishing gradients, their causes, and standard mitigations.
6. Select and justify an appropriate optimizer for a given real-world task.

1. Environment Setup and Data Exploration

1.1 Package Imports

Copy and run the following cell at the top of your notebook. If any import fails, install the missing package with `pip install <package>` before proceeding.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
```

```
import warnings
warnings.filterwarnings('ignore')

import tensorflow as tf
import keras
from keras import layers, models, callbacks
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

import matplotlib as mpl
mpl.rcParams['font.family'] = 'Times New Roman'
sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

print('Keras version    :', keras.__version__)
print('TensorFlow version:', tf.__version__)
```

1.2 Load the Dataset

```
CSV_PATH = 'Family Income and Expenditure.csv'    # update path as
    needed
df = pd.read_csv(CSV_PATH)
print('Shape:', df.shape)
df.head()
```

Run `df.info()` and `df.describe().T` to inspect data types, missing values, and summary statistics before answering the questions below.

Reflection Question 1

What does each *row* in the FIES dataset represent? Identify at least two demographic columns and two expenditure columns visible in `df.head()`.

Answer space

Each row is **one household**, not one person. The income and spending numbers are yearly totals for the whole family.

- **Demographic columns** in df.head(): Region and Main Source of Income (also Agricultural Household indicator). Further right in the full table there are also Household Head Sex and Household Head Age.
- **Expenditure columns** in df.head(): Total Food Expenditure and Bread and Cereals Expenditure (also Total Rice Expenditure, Meat Expenditure, Fruit Expenditure, and more).

The data has 41,544 households and 60 columns.

Reflection Question 2

The prediction target is **Total Food Expenditure**. Which column(s) should be *excluded* from the feature set to prevent data leakage? Justify your choice.

Answer space

Two columns are left out: **Total Food Expenditure** and **Total Household Income** (the sheet's Total Income).

1. **Total Food Expenditure** is the target. If it stays in the inputs, the model just copies the answer and the scores mean nothing.
2. **Total Household Income** is left out because income and food spending go up together very closely, so the model would depend on it almost completely, and a forecast should not need the family's income.

The food-group columns (rice, bread and cereals, meat, fish, fruit, vegetables) are parts of food spending. They stay in the baseline because the sheet sets the exclusion list to these two columns. Removing them lowers R^2 from 0.95 to 0.80, so they carry a large share of the prediction.

2. Data Preparation

Implement the following steps in your notebook to prepare your features and targets, split the data, and scale it.

Instructions

1. Set the prediction target to **Total Food Expenditure**, and define an exclusion list containing the target itself and **Total Income** (to avoid data leakage).
2. From the dataframe, select only the numeric columns, then remove the excluded columns from that list — this becomes your feature set.
3. Build your feature matrix from the selected numeric columns, filling any missing values with the *median* of each respective column. Do the same for the target column (filling missing values with its median).
4. Split the data into training and test sets using an **80/20** split. Fix the random seed to **42** so the split is reproducible.
5. Create a `StandardScaler` for the input features. *Fit* it on the training features only, then use it to *transform* both the training and test features. Fitting on the training set alone (rather than the full dataset) prevents information from the test set from leaking into the scaling statistics.
6. Create a *separate* `StandardScaler` for the target variable. Fit and transform it on the training target values only (reshaped into a single column) — this gives you the scaled training target used during model fitting.
7. Print the shapes of the resulting scaled training and test feature arrays to confirm the split and scaling were applied correctly.

2.1 — Questions

(a) Why is `StandardScaler` applied to the *features* before training a neural network?

(b) The scaler is `fit_transform`-ed on the training set but only `transform`-ed on the test set. Why is this distinction critical? What would go wrong if you fit the scaler on the entire dataset?

(c) Why is the *target* variable also scaled, and why must predictions be inverse-transformed before computing RMSE and R^2?

Answer space

(a) The columns are on very different sizes. Money columns are in the tens or hundreds of thousands of pesos, but counts like family size or number of rooms are small. With raw values, the big columns take over the weight updates just because of their units. StandardScaler gives every column mean 0 and standard deviation 1, so all inputs are on the same scale and training is faster and smoother.
(b) The scaler gets its mean and standard deviation from the **training rows only**, and the test rows are changed using those same numbers. The test set should act like data the model has never seen. If I fit on everything, the test rows would help set the numbers, so some test information leaks into training and the scores look better than they really are.
(c) The target is in the tens of thousands of pesos, so the starting errors and the squared loss are huge. Scaling it keeps the loss small and training steady. Then the model predicts in *scaled* units, but y_test is in pesos. So I use scaler_y.inverse_transform on the predictions first. After that, RMSE is the typical miss **in pesos**, and R^2 compares predictions and real values in the same units.

3. Model Architecture

Implement a model-building function in your notebook as described below. The same architecture will be used for all optimizers — this is a deliberate design choice.

Instructions

1. Write a function that accepts the number of input features (`input_dim`) and returns a sequential neural network.
2. The network's input layer should accept vectors of length `input_dim`.
3. Add a first hidden **dense** layer with **64 units** and **ReLU** activation.
4. Add a second hidden **dense** layer with **32 units** and **ReLU** activation.
5. Add an **output** dense layer with a single unit and *no* activation function (i.e., a linear output), since this is a regression task.
6. Instantiate the model using the number of columns in your scaled training feature set, and display its summary — this should list each layer's name, output shape, and number of trainable parameters.

3.1 — Architecture Analysis

(a) Calculate the *total number of trainable parameters* in the model. Show your work using the formula: `params = (input_dim + 1) × units`.

(b) Why does the output layer use a *linear* (no) activation instead of ReLU or sigmoid for this regression task?

(c) Holding the same architecture and the same random seed (`SEED = 42`) for all experiments is essential to fair comparison. Explain the principle behind this *controlled experiment* design.

Answer space

(a) There are 43 input features.

Layer	Work	Parameters
hidden_1	$(43 + 1) \times 64$	2,816
hidden_2	$(64 + 1) \times 32$	2,080
output	$(32 + 1) \times 1$	33
Total		**4,929**

This matches model.count_params() . The first layer has about 57% of all the parameters because each of the 43 inputs connects to all 64 units.

(b) The model has to be able to predict any real number. ReLU never goes below 0 and sigmoid stays between 0 and 1. The target was standardized, so a household that spends less than average has a *negative* scaled value, and a ReLU output could never give that. A linear output can be any number.

(c) Change only one thing at a time. If the architecture or the starting weights also changed between runs, I could not tell if a better RMSE came from the optimizer or just from a lucky start. SEED = 42 gives every optimizer the same starting weights and the same batch shuffling, so the optimizer is the only difference. Each optimizer is run only once, so the comparison is for this seed only.

4. Keras Optimizers

In your notebook, register all 13 optimizers listed below (for example, as entries in a dictionary mapping each optimizer's name to a configured Keras optimizer instance), then print a confirmation of how many optimizers were registered.

Optimizer	Configuration
SGD	Learning rate 0.01, momentum 0.9, Nesterov momentum enabled
RMSprop	Learning rate 0.001
Adam	Learning rate 0.001
AdamW	Learning rate 0.001, weight decay 0.004
Adadelta	Learning rate 0.001
Adagrad	Learning rate 0.001
Adamax	Learning rate 0.001
Adafactor	Learning rate 0.001
Nadam	Learning rate 0.001
Ftrl	Learning rate 0.001
Lion	Learning rate 0.0001 (note: 10$\times$ lower than the others)
Lamb	Learning rate 0.001
Muon	Learning rate 0.001

4.1 — Pre-training Prediction

Before running the training loop, predict the rankings of at least **four** optimizers. State which you expect to converge fastest, which might overfit, and which might underperform—and briefly explain your reasoning based on what you know from lectures.

Answer space

My predicted ranking by test RMSE, best to worst: **Adam, Nadam, SGD with Nesterov, then Adagrad.**
Adam and Nadam will converge fastest. From lecture, they scale each weight's step using its recent gradients, so their loss will drop sharply in the first 10 epochs. SGD uses the same learning rate for every weight, so it will be slower even with momentum.
Adam will overfit the most. It fits the training set so fast that its validation loss will start going up while the training loss keeps going down, and EarlyStopping will stop it. The model has only 4,929 parameters, so the gap will be small.
Adagrad will do the worst. It keeps adding up squared gradients, so its steps keep getting smaller, and with a learning rate of 0.001 it will stall before it fits well.

5. Training Loop

Implement a training loop in your notebook that trains and evaluates the model once for each of the 13 optimizers. Training all optimizers will take several minutes.

Instructions

1. Set **EPOCHS = 80**, **BATCH = 64**, and **SEED = 42** as constants, and fix the global random seed to `SEED` before the loop begins.
2. Create empty containers to hold, for each optimizer: its training history, its evaluation metrics, and the trained model object itself.
3. For *each* optimizer in your registered dictionary, repeat the following:
   (a) Re-fix the random seed to `SEED` so every optimizer starts from the same initial weights.
   (b) Build a fresh instance of the model using your model-building function from Section 3.
   (c) Compile the model with the current optimizer and **mean squared error (MSE)** as the loss function.
   (d) Configure an **EarlyStopping** callback that monitors validation loss, uses a patience of 15 epochs, and restores the best weights seen during training.
   (e) Fit the model using a **15% validation split** taken from the training data, the configured number of epochs and batch size, and the EarlyStopping callback. Store the returned training history.
   (f) Generate predictions on the scaled test features, then inverse-transform these predictions back to the original expenditure units using the target scaler from Section 2.
   (g) Compute the **RMSE** and $\mathbf{R^2}$ between the inverse-transformed predictions and the actual (unscaled) test targets.
   (h) Record how many epochs the model actually trained for (it may stop before reaching 80 due to early stopping), along with the final training loss and final validation loss — both rescaled back to the original expenditure units (by multiplying the scaled loss by the square of the target scaler's scale factor).
   (i) Print a one-line summary showing the optimizer's name, RMSE, R^2, and number of epochs run.

Note on EarlyStopping: The callback monitors validation loss with `patience=15` and restores the best weights. This means different optimizers may run a different number of epochs—`Epochs Run` in your results table will reflect this.

5.1 Record Your Results

After training completes, fill in the table below with your actual results.

Optimizer	RMSE	R^2	Train Loss	Val Loss	Epochs
SGD	11,482.93	0.9504	5.30×10^7	1.07×10^8	76
RMSprop	11,355.11	0.9515	6.44×10^7	1.12×10^8	20
Adam	11,163.13	0.9531	5.78×10^7	1.04×10^8	25
AdamW	11,264.44	0.9523	5.37×10^7	1.09×10^8	34
Adadelta	27,128.44	0.7232	6.74×10^8	6.92×10^8	80
Adagrad	13,129.63	0.9352	1.25×10^8	1.57×10^8	80
Adamax	11,274.33	0.9522	5.78×10^7	9.91×10^7	57
Adafactor	11,171.56	0.9531	6.02×10^7	9.98×10^7	80
Nadam	11,205.56	0.9528	5.42×10^7	1.12×10^8	30
Ftrl	11,206.14	0.9528	8.34×10^7	9.72×10^7	80
Lion	10,942.84	0.9550	5.59×10^7	1.01×10^8	23
Lamb	11,254.79	0.9524	5.75×10^7	1.04×10^8	29
Muon	11,223.06	0.9526	5.16×10^7	1.09×10^8	22

RMSE and R^2: test set, original pesos. Train Loss and Val Loss: MSE of the last epoch that ran, in original units (pesos^2). Each optimizer was run only once (seed 42). Early stopping brought back the best-epoch weights.

6. Visualizations

6.1 Per-Optimizer Diagnostic Plots

For each optimizer, generate one figure containing three side-by-side subplots.

Instructions

1. For the current optimizer, retrieve its stored training history, metrics, and trained model from Section 5.
2. Generate predictions on the scaled test set with that model, inverse-transform them to the original units, and compute the residuals (actual minus predicted).
3. Create a figure with **three subplots arranged in one row**:
   (a) **Loss curve** — plot the training loss and validation loss (rescaled to original expenditure units) against epoch number, with a legend distinguishing the two curves, and title the subplot with the optimizer's name.
   (b) **Predicted vs. actual scatter plot** — plot predicted expenditure against actual expenditure for every test sample, overlay a diagonal reference line representing a perfect fit, and annotate the subplot with the optimizer's R^2 and RMSE values.
   (c) **Residual histogram** — plot a histogram of the residuals (40 bins), with a vertical reference line at zero.
4. Give the overall figure a title naming the optimizer, and save it as an image file named after that optimizer (e.g., `fig_<optimizer_name>.png`).

5. Repeat this process for all 13 optimizers.

6.2 Comparative Plots

Instructions

1. Create a figure with **two side-by-side subplots**: one plotting every optimizer's *training* loss curve across epochs, and the other plotting every optimizer's *validation* loss curve across epochs. Label each curve with its optimizer's name and include a legend on both subplots.
2. Save this combined figure as `fig_all_loss_curves.png`.
3. Build a **horizontal bar chart** ranking all 13 optimizers by test RMSE, sorted from best (lowest RMSE) to worst (highest RMSE). Color the bars on a gradient (e.g., green for best, red for worst) and label each bar with its numeric RMSE value.
4. Title the chart appropriately and save it as `fig_rmse_comparison.png`.

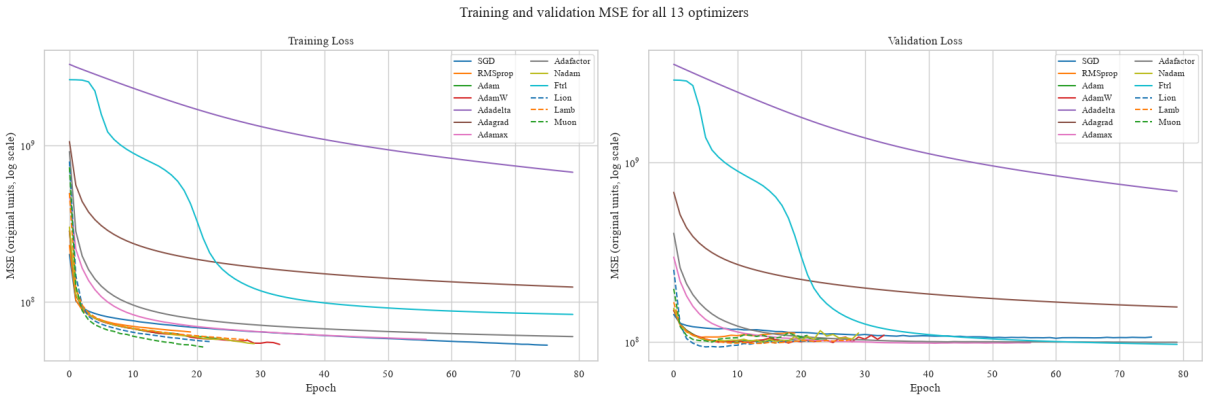


Figure 1: Training (left) and validation (right) MSE in original units for all 13 optimizers, on a log scale.

The 13 per-optimizer diagnostic figures (loss curves, predicted vs. actual, residual histogram) are in Appendix B.

7. Discussion Questions

Instructions: Answer all questions in the spaces provided. Support quantitative claims with values from your results table. Use mathematical notation where appropriate.

Question 1 — Backpropagation Mechanics (15 pts)

Consider the 3-layer network in this activity:

$$x \xrightarrow{W^1, b^1} z^1 \xrightarrow{\text{ReLU}} a^1 \xrightarrow{W^2, b^2} z^2 \xrightarrow{\text{ReLU}} a^2 \xrightarrow{W^3, b^3} \hat{y}$$

(a) Write the *forward pass* equations for each layer (pre-activation and post-activation).

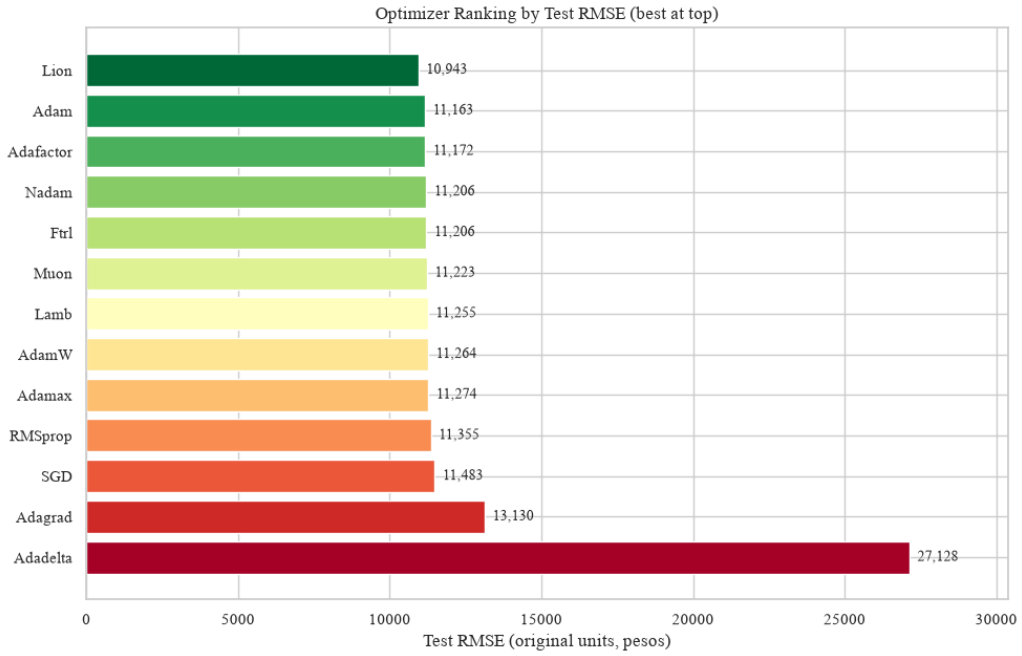


Figure 2: Optimizers ranked by test RMSE (original units), best at top.

(b) Define the Mean Squared Error loss: $L = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$. Derive $\dfrac{\partial L}{\partial W^3}$ using the chain rule, showing each intermediate gradient.
(c) Explain in words why backpropagation is more efficient than computing each weight's gradient independently (finite-difference approximation). Reference the number of parameters in your model.

Answer space

(a) Forward pass. For one input row $x \in \mathbb{R}^{1\times 43}$:

$$z^1 = xW^1 + b^1, \quad W^1 \in \mathbb{R}^{43\times 64} \qquad a^1 = \max(0, z^1)$$
$$z^2 = a^1 W^2 + b^2, \quad W^2 \in \mathbb{R}^{64\times 32} \qquad a^2 = \max(0, z^2)$$
$$\hat{y} = a^2 W^3 + b^3, \quad W^3 \in \mathbb{R}^{32\times 1}$$

There is no activation on the last layer, so $\hat{y} = z^3$.
(b) Finding $\partial L/\partial W^3$. The chain is $L \to \hat{y}_i \to W^3$.
Step 1. Loss with respect to one prediction: $\dfrac{\partial L}{\partial \hat{y}_i} = \dfrac{2}{n}(\hat{y}_i - y_i)$.
Step 2. Prediction with respect to W^3. Since $\hat{y}_i = a_i^2 W^3 + b^3$ and a_i^2 does not depend on W^3: $\dfrac{\partial \hat{y}_i}{\partial W^3} = (a_i^2)^T \in \mathbb{R}^{32\times 1}$.
Step 3. All samples share W^3, so I multiply and add over i:

$$\frac{\partial L}{\partial W^3} = \sum_i \frac{\partial L}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W^3} = \frac{2}{n}\sum_i (a_i^2)^T(\hat{y}_i - y_i) = \frac{2}{n}(A^2)^T(\hat{y} - y)$$

A^2 is the $n \times 32$ matrix of second-layer activations and $\hat{y} - y$ is $n \times 1$. The product is 32×1, the same shape as W^3. For the bias, $\partial \hat{y}_i / \partial b^3 = 1$, so $\dfrac{\partial L}{\partial b^3} = \dfrac{2}{n} \sum_i (\hat{y}_i - y_i)$. I coded these formulas in NumPy for one batch of 64 rows. They match TensorFlow's automatic gradients (biggest difference 7×10^{-7}, which is float32 rounding) and a finite-difference estimate for all 4,929 parameters (biggest difference 8×10^{-10}).

(c) Why backpropagation is cheaper. A finite-difference gradient moves **one** weight by a tiny ϵ, runs the forward pass again, and uses $\dfrac{L(\theta_j + \epsilon) - L(\theta_j - \epsilon)}{2\epsilon}$. My model has 4,929 parameters, so getting the whole gradient takes **9,858 extra forward passes** (two per parameter), and this has to be done for every batch of every epoch. Picking ϵ is also tricky: if it is too big the answer is not accurate, and if it is too small it gets lost in rounding error.

Backpropagation does **one forward pass and one backward pass** and gets all 4,929 gradients. That costs about the same as two or three forward passes. It works because the error signal is reused: $\delta^3 = \frac{2}{n}(\hat{y} - y)$ gives $\partial L / \partial W^3$ and is then passed back with $\delta^2 = (\delta^3 (W^3)^T) \odot \mathrm{ReLU}'(z^2)$, so no layer starts from scratch. I timed one batch on this CPU: backprop took 1.08 ms and finite differences took 355 ms, so finite differences were about **328 times slower**.

Question 2 — Loss Landscape Observations (10 pts)

(a) From your comparative loss-curve plot, which optimizer(s) descended fastest in the first 10 epochs? Which descended slowest? Cite specific epoch numbers and approximate loss values from your plot.

(b) Did any optimizer's *validation* loss increase while its *training* loss continued to decrease? If so, name the optimizer and the approximate epoch where divergence began. What does this indicate?

(c) Explain, using the concept of *per-coordinate adaptive step sizes*, why adaptive optimizers (Adam, RMSprop) tend to show faster early descent on tabular datasets like FIES.

Answer space

Epoch numbers start at 0, like on the plots. Loss values are MSE in original units, read from the table above.

(a) Fastest and slowest. By epoch 10, the lowest training losses were **Muon (6.0×10^7)**, **Lion (6.4×10^7)**, AdamW (6.8×10^7) and Nadam (6.8×10^7). Muon went from 7.3×10^8 at epoch 0 to 7.7×10^7 at epoch 3, and Lion went from 7.9×10^8 to 8.1×10^7. The other adaptive optimizers (RMSprop, Adam, AdamW, Nadam, Lamb) were between 8.3×10^7 and 8.6×10^7 at epoch 3 and stayed close together after epoch 10.

Adadelta was the slowest. It went from 3.3×10^9 at epoch 0 to only 2.3×10^9 at epoch 10, and it was still 6.7×10^8 at epoch 79, which is higher than Adam's loss at epoch 0 (2.8×10^8). Ftrl (8.9×10^8 at epoch 10) and Adagrad (2.4×10^8 at epoch 10) were also slow. (SGD looks like it started low, at 2.0×10^8, because the epoch-0 loss is an average over the whole first epoch.)

(b) Validation up while training down. Yes, for several runs. The clearest cases (validation loss up by more than 5% after its best epoch, while training loss kept going down) were:

Optimizer	Best epoch	Val loss rise afterwards	Train loss change
Nadam	14	+13.1%	-15.8%
AdamW	18	+10.6%	-12.6%
Muon	6	+9.0%	-22.0%
Lion	7	+7.7%	-18.1%
Lamb	13	+5.7%	-11.9%
RMSprop	4	+5.6%	-18.6%

Adam, SGD, Adamax and Adafactor had smaller rises (0-5%). Adadelta, Adagrad and Ftrl were still improving at epoch 79, so no divergence. This is the **start of overfitting**: the model keeps getting better on the training data but stops getting better on data it has not seen. EarlyStopping stopped these runs and brought back the best weights, which is why their test RMSE ended up close to everyone else's. (I defined divergence as a validation rise of more than 5% after the best epoch. Training loss is an average over the epoch, while validation loss is measured at the end.)

(c) Adaptive step sizes for each weight. Adam and RMSprop keep a running average of the squared gradient *for each weight*, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$, and divide that weight's step by $\sqrt{v_t}$:

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}$$

So each of the 4,929 parameters gets its own step size. This matters for FIES because the 43 features are very different. 14 of the 43 are zero in more than half of the households (for example, *Number of Car, Jeep, Van* is zero in 93.6%), so their weights get small gradients once in a while, while the weights on dense columns get big gradients. SGD uses one learning rate for all of them, so it has to be small enough for the biggest gradients, and that makes it slow for the rest. Dividing by $\sqrt{v_t}$ gives bigger steps to weights with small gradients and smaller steps to weights with big gradients, so every weight moves by about η per step. That matches the plot, where the adaptive group drops to about 8×10^7 within 3 epochs.

Question 3 — Adaptive vs. Non-Adaptive Optimizers (10 pts)

(a) Complete the comparison table below.

Dimension	SGD (Nesterov)	Adam
Update rule (equation)	$v_t = \mu v_{t-1} - \eta g_t$ $\theta_{t+1} = \theta_t + \mu v_t - \eta g_t$	$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$ with $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$, $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$
Auxiliary state	One extra vector: the velocity v_t (momentum).	Two extra vectors: m_t (average gradient) and v_t (average squared gradient).
LR sensitivity	High: all weights share one η, so it must be tuned carefully (too large: the loss jumps around; too small: very slow).	Lower: each weight's step is divided by $\sqrt{\hat{v}_t}$, so η = 0.001 worked without tuning.
Memory overhead	1 stored number per parameter (4,929 extra here, measured).	2 stored numbers per parameter (9,858 extra here, measured).

(b) In your results, did SGD perform better or worse than Adam? At what epoch count did SGD stop (EarlyStopping)? Speculate why a higher or lower `patience` value might have changed SGD's outcome.

Answer space

(a) The comparison table above is filled in. I checked the update equations against Keras; the largest difference was due to float32 rounding.
(b) Adam performed better than SGD in this run. Adam had RMSE **11,163.13** and R^2 0.9531, while SGD had RMSE **11,482.93** and R^2 0.9504. Adam stopped after 25 epochs (best epoch 9). SGD stopped after 76 epochs (best epoch 60). The patience test gave RMSE 11,696.42 at patience 5, and RMSE 11,482.93 at patience 15 and 30. Patience 5 stopped at epoch 43 with best epoch 37; patience 15 stopped at epoch 76 with best epoch 60; patience 30 reached the 80-epoch limit with the same best epoch and score.
A higher patience would not have changed SGD's result, because its best epoch (60) was already found with patience 15. A lower patience hurt it: with patience 5, SGD stopped at epoch 43, before it reached its best epoch of 60, and its RMSE was worse. SGD improves slowly, so it needs a longer patience than Adam, which found its best epoch at 9.

Question 4 — The Adam Optimizer Family (15 pts)

Four of your optimizers are members of the Adam family: **Adam**, **AdamW**, **Adamax**, and **Nadam**.
(a) For each optimizer, write its update rule and describe in one sentence how it differs from baseline Adam.
(b) Among these four, which achieved the lowest RMSE in your results? Which had the highest R^2? Are they the same optimizer? Discuss what this tells you.

(c) AdamW decouples weight decay from the gradient update. Write both (i) the Adam+L2 update and (ii) the AdamW update, and explain the key difference. Why does the distinction matter in practice?

Answer space

(a) All four use the same running averages:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

- **Adam (baseline):** $\theta_{t+1} = \theta_t - \eta \dfrac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$.
- **AdamW:** $\theta_{t+1} = \theta_t - \eta \left(\dfrac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$ with $\lambda = 0.004$. It is different from Adam because it shrinks the weights directly, outside the gradient.
- **Adamax:** $u_t = \max(\beta_2 u_{t-1}, |g_t|)$, $\theta_{t+1} = \theta_t - \dfrac{\eta}{1-\beta_1^t} \dfrac{m_t}{u_t + \epsilon}$. It uses the biggest recent gradient size instead of the average of the squared gradients.
- **Nadam:** $\theta_{t+1} = \theta_t - \dfrac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \left(\beta_1 \hat{m}_t + \dfrac{(1-\beta_1) g_t}{1-\beta_1^t} \right)$. It adds a Nesterov look-ahead by mixing the current gradient into the momentum. (Keras also uses a momentum schedule, so this is the standard simplified form, and the very first Keras step is about 0.56 times the size of this formula's.)

The Adam, AdamW, Adamax and Lion equations match Keras to 3×10^{-7} or better in the notebook check.

(b)

Optimizer	RMSE	R^2	Epochs
Adam	11,163.13	0.9531	25
Nadam	11,205.56	0.9528	30
AdamW	11,264.44	0.9523	34
Adamax	11,274.33	0.9522	57

Adam has both the lowest RMSE and the highest R^2, so it is the same optimizer. This is not a coincidence: on one test set $R^2 = 1 - \text{MSE}/\text{Var}(y)$, so a lower RMSE always means a higher R^2, and the two rankings have to agree (the notebook rebuilds R^2 from RMSE to within 3×10^{-8}). What matters more is how small the gaps are: the whole family is within **1.00%**. So the four optimizers do about the same on this dataset. Look-ahead, max-norm scaling and separate weight decay change the RMSE by about 1%. Adamax ran longest at 57 epochs, but did not give the best RMSE.

(c) With **L2 regularization**, the penalty $\frac{\lambda}{2}\|\theta\|^2$ is part of the loss, so $\lambda\theta_t$ is added to the gradient *before* the moments are computed:

$$\text{(i)} \quad g'_t = g_t + \lambda \theta_t, \qquad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}'_t}{\sqrt{\hat{v}'_t} + \epsilon}$$

where $\hat{m}'_t, \hat{v}'_t$ are built from g'_t. **AdamW** puts the decay straight on the weights:

$$\text{(ii)} \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta\lambda\theta_t$$

Key difference: in (i) the decay term is divided by $\sqrt{\hat{v}_t'}$ like everything else, so a weight with large gradients is decayed *less* and a weight with small gradients is decayed *more*. The regularization becomes uneven and tangled up with the adaptive step sizes. In (ii) every weight shrinks by the same factor $\eta\lambda$ no matter how big its gradient is, so λ can be tuned separately from the gradient size, and it usually does a bit better on new data. In my run the effect was tiny: AdamW (11,264.44) and Adam (11,163.13) differ by about 0.91%. With $\lambda = 0.004$ and only 4,929 parameters, there was not much overfitting for the decay to fix.

Question 5 — Vanishing Gradients (10 pts)

(a) Show mathematically why gradients can vanish in deep networks. Write the general form of $\frac{\partial L}{\partial W^1}$ for a network with L layers and activation σ. Under what conditions does this product shrink to near zero?
(b) The model uses **He initialization** internally via Keras defaults for ReLU layers. State the He initialization formula for weight variance. Explain intuitively why this initialization reduces the vanishing gradient problem at the start of training.
(c) Suppose you replaced all ReLU activations with **sigmoid**. Based on the maximum derivative of the sigmoid function, explain whether you would expect training to be more or less stable, and why.

Answer space

(a) Let layer l do $z^l = a^{l-1}W^l + b^l$ and $a^l = \sigma(z^l)$, with $a^0 = x$. Going backward, each layer passes the error back by multiplying by that layer's weights and by σ':

$$\delta^l = \left(\delta^{l+1}(W^{l+1})^T\right) \odot \sigma'(z^l), \qquad \frac{\partial L}{\partial W^1} = x^T\delta^1$$

Written all the way back from the output:

$$\frac{\partial L}{\partial W^1} = x^T\,\delta^L \prod_{l=L-1}^{1} \left[(W^{l+1})^T \operatorname{diag}\left(\sigma'(z^l)\right)\right]$$

This is a product of many factors. With $\gamma = \max|\sigma'|$:

$$\left\|\frac{\partial L}{\partial W^1}\right\| \le \|x\|\,\|\delta^L\| \prod_{l=2}^{L} \|W^l\|\,\gamma$$

If $\|W^l\|\gamma < 1$ for most layers, the product shrinks **exponentially** with depth, so the early layers barely learn. This happens when σ' is small (the flat parts of sigmoid or tanh) or when the weights are small.

(b) He initialization: $\operatorname{Var}(W) = \frac{2}{n_{in}}$, where n_{in} is the number of inputs to the layer. ReLU turns about half of the values into zero, so the variance of the signal is cut

about in half at every layer. The factor 2 makes up for that, so the size of the signal (and of the gradients going backward) stays about the same from layer to layer. ReLU's slope is also exactly 1 for positive inputs, so the σ' factors in (a) do not shrink anything.

Correction to the sheet: Keras Dense layers do **not** use He initialization by default. The default is **Glorot uniform**, $\mathrm{Var}(W) = \dfrac{2}{n_{in} + n_{out}}$. The notebook prints GlorotUniform and measures the variances (Glorot 0.0184 vs formula 0.0187; He 0.0475 vs formula 0.0465 for the $43 \to 64$ layer). So my baseline uses Glorot.

Evidence for the idea (10 hidden layers of 32 ReLU units, standard deviation of the activations from layer 1 $\to$ layer 10): Glorot **0.653** $\to$ **0.022** (the signal gets about 30 times smaller), He **1.047** $\to$ **1.044** (stays the same). For my real model with 2 hidden layers, the choice hardly matters: Adam with He init gave RMSE 11,234.41 and with Glorot it gave 11,163.13, a gap of about 0.64%.

(c) The sigmoid derivative is $\sigma'(z) = \sigma(z)(1-\sigma(z))$, which is **never more than 0.25** (at $z = 0$; I checked it with numbers). So every sigmoid layer cuts the backward gradient to at most a quarter. With my two hidden layers the first-layer gradient would be at most $0.25^2 = 0.0625$ of its size, and with ten layers $0.25^{10} \approx 10^{-6}$. In the 10-layer test, the ratio of the first-layer gradient to the last-layer gradient was about 10^{-5} for sigmoid but 0.95 for ReLU with Glorot.

I would expect training to be **less stable and slower**, especially in deeper networks, because sigmoid derivatives repeatedly shrink the gradients passed to earlier layers. This is a vanishing-gradient problem, not a numerical blow-up. In my network with two hidden layers, the derivative contribution is at most $0.25^2 = 0.0625$, so the effect is milder here than it would be in a deeper network.

Question 6 — Adafactor vs. Adam (10 pts)

(a) Adam stores a full second-moment tensor $v_t \in \mathbb{R}^{n \times m}$ for every weight matrix. Explain how Adafactor approximates this tensor using row and column vectors r_t and c_t. What is the memory complexity of each approach?

(b) In your results, how did Adafactor compare to Adam in terms of RMSE, R^2, and epochs run? Were the results similar or notably different? Explain your observation.

(c) For which type of model or application is Adafactor's memory advantage most significant? Why would it not matter much for this FIES MLP?

Answer space

(a) Adam keeps a running average of the squared gradient for **every** weight, so for a matrix $G_t \in \mathbb{R}^{n \times m}$ it stores $v_t \in \mathbb{R}^{n \times m}$. Adafactor keeps only one number per row and one per column:

$$r_t = \beta_2 r_{t-1} + (1 - \beta_2)(G_t^2)\mathbf{1}_m, \qquad c_t = \beta_2 c_{t-1} + (1 - \beta_2)\mathbf{1}_n^T(G_t^2)$$

with $r_t \in \mathbb{R}^n$, $c_t \in \mathbb{R}^m$. It builds the full matrix back from the two:

$$\hat{V}_t = \frac{r_t c_t}{\mathbf{1}_n^T r_t}, \qquad \theta_{t+1} = \theta_t - \eta \frac{G_t}{\sqrt{\hat{V}_t} + \epsilon}$$

Each entry is estimated from how big its row and its column are, instead of being tracked by itself. **Memory:** Adam $O(nm)$, Adafactor $O(n + m)$. For my first layer (43×64) that is 2,752 vs $43 + 64 = 107$, and for the second layer (64×32) it is 2,048 vs 96. For the whole model (biases are not factored) the second-moment storage is **4,929 values for Adam vs 333 for Adafactor**.

(b) The results were close, and Adafactor was slightly better:

	RMSE	R^2	Epochs	Best epoch
Adafactor	11,171.56	0.9531	80	75
Adam	11,163.13	0.9531	25	9

The RMSE gap is 0.08% in Adafactor's favor, so both reach almost the same accuracy. The **epochs** were very different. Adam's validation loss stopped improving at epoch 9, so EarlyStopping ended it at 25. Adafactor's validation loss kept improving slowly until epoch 75, so early stopping never kicked in and it used all 80 epochs. Adafactor's training loss fell fast at first (86.2% of the starting loss was gone by epoch 5), but its validation loss crept down for much longer, so it needed many more epochs to reach about the same accuracy.

(c) The memory saving matters most for **very large models** (transformers, language models), where the weight matrices are huge and Adam's extra tensors can be as big as the model itself. It does not matter for my FIES model: it has 4,929 parameters, about 19 KB, so Adam's extra storage is tiny and memory is not a problem. If anything, Adafactor just needed more epochs here.

Question 7 — Lion Optimizer (10 pts)

Lion is an optimizer discovered via *learned optimization* rather than hand-crafted rules. Its learning rate was set to `0.0001` ($10\times$ lower than the other optimizers in this experiment).

(a) Write Lion's update rule. What distinguishes Lion's update from Adam's update in terms of the magnitude of each weight change?

(b) Why does Lion's learning rate need to be set substantially lower than Adam's? What property of Lion's update makes a large learning rate dangerous?

(c) Based on your results, did Lion perform well on this tabular regression task? Propose one reason—related to gradient noise or dataset size—why Lion's sign-based update might underperform on FIES compared to a vision dataset.

Answer space

(a) Lion keeps one momentum value m_t per weight and moves each weight by the sign of a blend of the momentum and the current gradient:

$$c_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad \theta_{t+1} = \theta_t - \eta \,\mathrm{sign}(c_t), \qquad m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t$$

Every weight moves by exactly η per step, whatever the size of its gradient. Adam's step, $\eta\, \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$, changes with the gradient history and is usually

smaller than η when gradients are noisy.
(b) Because Lion uses the sign, every one of the 4,929 weights moves by the full η each step, even when its gradient is tiny. A large η therefore makes every weight jump by a large fixed amount, and the loss can overshoot. In the notebook's demonstration, both optimizers used a learning rate of 0.001: Lion moved exactly 0.001 per step, while Adam's average step was only about 0.000169. Setting Lion's rate 10 times lower (0.0001) keeps its steps closer to Adam's typical size.
(c) Lion performed best in the latest run. It achieved RMSE **10,942.84**, R^2 **0.9550**, and ranked **1st of 13**. It trained for 23 epochs and reached its best epoch at 7. Its RMSE was about 2.0% lower than Adam's 11,163.13. This makes Lion the strongest optimizer by test RMSE in this run.
The sign update could still underperform on FIES for one reason: gradient noise. FIES has about 28,000 training rows after the validation split, and mini-batch gradients on this tabular data are noisy. The sign discards the size of the gradient, so a noisy, unreliable gradient moves a weight by the same amount as a reliable one. A large vision dataset gives cleaner averaged gradients, so the sign is more reliable there. Lion's validation loss rose 7.7% after its best epoch (epoch 7), which fits this reason.

Question 8 — Overfitting and Regularization (10 pts)

(a) From your results table, identify the optimizer with the *largest gap* between Final Train Loss and Final Val Loss. Is this evidence of overfitting? Explain using the values you recorded.
(b) List and briefly explain **three** regularization techniques you could apply to reduce overfitting in this model. State the specific Keras API call or parameter you would modify for each.
(c) The training loop uses `EarlyStopping`. Is EarlyStopping a form of regularization? Justify your answer. What risk does it introduce if `patience` is set too low?

Answer space

(a) Nadam had the largest gap between final validation loss and final training loss: $1.12 \times 10^8 - 5.42 \times 10^7 \approx 5.8 \times 10^7$. Muon was very close at 5.7×10^7 and had the largest ratio (2.11, against 2.07 for Nadam). This is mild overfitting. Nadam's best validation loss was at epoch 14, and after that its validation loss rose 13.1% while its training loss fell 15.8%. The validation loss is about twice the training loss, but the test RMSE of 11,205.56 is only 2.4% above Lion, the best optimizer, so the overfitting is small. Adadelta had the smallest ratio (1.03), but both of its losses were high, which means underfitting, not good generalization.
(b) Three ways to reduce overfitting:

1. **Dropout** randomly turns off some hidden units during training. In Keras I would add `layers.Dropout(0.2)` after each hidden Dense layer in the model-building function.
2. **L2 regularization** adds a penalty for large weights to the loss. In Keras I would set `kernel_regularizer=regularizers.l2(1e-4)` in each hidden

`layers.Dense`.

3. **Weight decay** gradually shrinks weights during optimization. In Keras I would raise the `weight_decay` argument of `keras.optimizers.AdamW`, for example from 0.004 to 0.01.

(c) Yes, early stopping is a form of regularization. It limits how long the model can fit the training data, so training stops before the model starts to memorize noise, and restoring the best weights keeps the model with the lowest validation loss. In this run, 9 of the 13 optimizers stopped early. The risk of a patience that is too low is stopping before the real best epoch. In the patience-5 test, SGD stopped at epoch 43 and got RMSE 11,696.42, compared with 11,482.93 at patience 15.

Question 9 — Muon Optimizer (10 pts)

Muon applies *Nesterov momentum* followed by an orthogonalization step (Newton–Schulz iteration) to 2-D weight matrices. For all other parameter types (biases, embeddings, and the output layer), it falls back to **AdamW**.

(a) List the specific variable types or layers that Muon routes to its internal AdamW optimizer. Why does it exclude these from the orthogonalization step?

(b) Based on your results, how did Muon compare to AdamW on the FIES task? Was any difference meaningful given the dataset size and model scale?

(c) According to the Keras documentation, what type of model benefits most from Muon, and why? Would you recommend Muon for a small tabular MLP deployed in production? Justify your answer.

Answer space

(a) According to the Keras documentation, Muon uses its special update only on **2-D weight matrices**. Three kinds of variables use the internal **AdamW** step instead: every variable that is not 2-D (all **biases**, and this cannot be changed), **embedding layers** (exclude_embeddings=True by default), and any layer listed in exclude_layers, where the documentation says to pass the name of the densely connected **output layer**. Orthogonalization only works on a matrix, and a bias is a vector, so there is nothing to orthogonalize. Muon was made for the hidden-layer weight matrices, while embeddings and the output layer connect to the input or the target, so AdamW handles them.

For my model : **Muon step:** hidden_1/kernel (2,752 values) and hidden_2/kernel (2,048) = 4,800 values. **AdamW:** hidden_1/bias, hidden_2/bias, output/kernel and output/bias = 129 values.

I also ran Muon without exclude_layers, so the output kernel used Muon too (S1): RMSE 11,157.84 (21 epochs) vs 11,223.06 (22 epochs) with the exclusion. The difference is small, but the version with the exclusion is the one that matches the documentation.

(b) Muon was very slightly better than AdamW (0.37% lower RMSE), but not by enough to matter:

	RMSE	R^2	Epochs	Train loss	Val loss
Muon	11,223.06	0.9526	22	5.16×10^7	1.09×10^8
AdamW	11,264.44	0.9523	34	5.37×10^7	1.09×10^8

Muon's final validation-to-training loss ratio was 2.11, the largest of the 13 optimizers, and AdamW's was 2.04. Muon fit the training data slightly harder (train loss 5.16×10^7 vs 5.37×10^7), but both ended with the same validation loss (1.09×10^8). With a 0.37% RMSE difference from one run, the difference is not meaningful for a dataset and model this small.

(c) The Keras documentation does not name a type of model. Its references are the original Muon code and the paper *Muon is Scalable for LLM Training*, which show that Muon was made for the hidden layers of large models like language models. Orthogonalizing the update of each big weight matrix spreads the update evenly across directions, which makes training more efficient at that size. My model has only 4,929 parameters and two small weight matrices, so there is not much to gain.

I do **not** recommend Muon for a small tabular MLP in production. It ranked 6th of 13 (RMSE 11,223.06), behind Lion and Adam, and had the largest final validation-to-training loss ratio. It also needs more work: extra calculations for every weight matrix and a choice about which layers to leave out. AdamW got the same accuracy with less setup.

Question 10 — Design Recommendation (10 pts)

A provincial government office in Mindanao wants to build a model to *predict household food expenditure* from survey data similar to FIES. They have limited compute resources (a single GPU) and need a deployable model within two weeks.

(a) Based on your experimental results, recommend **one optimizer** for this deployment scenario. Justify your choice using at least three criteria (e.g., convergence speed, final RMSE, regularization, ease of tuning).

(b) List the **first two hyperparameters** you would tune for your chosen optimizer, and specify a reasonable search range for each.

(c) The survey data contains features at very different scales (e.g., annual income in millions of pesos vs. household size of 1–15). Explain how your chosen optimizer handles heterogeneous feature scales, and whether any additional preprocessing step is advisable.

Answer space

(a) I recommend **Lion**, for three reasons:

1. **Final RMSE:** it had the lowest test RMSE (**10,942.84**) and highest R^2 (**0.9550**) of the 13 optimizers, about 2.0% better than Adam (11,163.13).
2. **Convergence speed:** it removed 90.9% of its starting training loss by epoch 5, reached its best validation loss at epoch 7, and stopped at epoch 23 (32 seconds in my run). That fits a two-week deadline and one GPU.
3. **Memory and setup:** it stores one extra value per parameter (4,929 here),

half of Adam's 9,858, and its main setting is the learning rate.
These results come from one run with seed 42.
The choice must also consider which columns will be available when predictions are made. Removing the food-group expenditure columns reduced Adam's R-squared to **0.8016** and SGD's to **0.8011**. The office must confirm that these columns will be available before using the model for real predictions.
(b) The first two hyperparameters I would tune are:

1. **Learning rate:** test values from 10^{-5} to 10^{-3} on a log scale, centered on the 0.0001 used here, because Lion needs a lower rate than Adam.
2. **Weight decay:** test values from 10^{-4} to 10^{-1} for optimizers that support it.

I would compare each setting using the same training and validation split, selecting the best settings by validation performance. I would keep the held-out test set untouched during tuning and use it only once for the final evaluation.
(c) Lion uses only the sign of its update, so every weight moves by the same step η whether its gradient is large or small. The size of a feature's gradient therefore does not change the step size. This does not fix the scale of the inputs themselves, so I would keep StandardScaler, fit it only on the training data, and check skewed features and outliers (such as income) before training. Any transformations must also be applied the same way when making predictions.

8. Convergence Analysis (Bonus — 10 pts)

Generate the normalised convergence plot described below and answer the accompanying question.

Instructions

1. For each optimizer, take its training loss curve and **normalize** it by dividing every value in the curve by the loss recorded at epoch 0 — the result is a curve that starts at 1.0 and tracks the fraction of the initial loss remaining at each subsequent epoch.
2. Plot all 13 normalized curves on a single chart, with each curve labeled by its optimizer's name, and restrict the y-axis to the range 0 to 1.2 for readability.
3. Title the chart *"Normalised Training Loss (loss / initial loss)"*, label the axes appropriately, and save the figure as `fig_convergence_normalised.png`.

Bonus Question — Normalised Convergence

(a) Normalizing by the initial loss removes the absolute scale. What does this allow you to compare fairly across optimizers?
(b) Identify the optimizer that reduced its normalised loss by the greatest fraction within the *first 5 epochs*. Explain why this specific optimizer converges so rapidly in the early phase.
(c) Which optimizer maintained the most *stable* convergence (smoothest curve without large oscillations)? Why might stability matter more than raw speed in a production setting?

Answer space

(a) Dividing each training-loss curve by its epoch-0 loss makes all curves start at 1.0. This makes it easier to compare how quickly each optimizer reduces its own starting loss. The limitation is that a lower normalized loss does not necessarily mean a lower final loss, so the test RMSE table is still needed.
(b) Lion reduced its normalized loss the most by epoch 5: **90.9%**. Muon was next at **90.5%**, followed by Adafactor at 86.2% and Lamb at 84.6%. Adadelta removed only 16.5%. Fast early loss reduction did not always mean the best final result, although Lion also achieved the best test RMSE.
(c) SGD had the lowest roughness score at **0.369**, followed by RMSprop at 0.552. Ftrl had the highest score at 5.586, but its training loss did not rise; its large score reflects its steep changes. Stability matters more than raw speed in production because a stable optimizer gives nearly the same model every time it is retrained on new survey data, and early stopping can pick the best epoch reliably. A fast but jumpy run can stop at a poor epoch and needs extra monitoring.

Submission Checklist

Before submitting, verify that you have:

- ☐ Completed all coding cells in your own notebook, following the given instructions.
- ☐ Filled in the Results Table in Section 5 with your actual values.
- ☐ Answered all 10 discussion questions (Q1–Q10) in this activity sheet.
- ☐ Attached or embedded the figures generated by your notebook (`e.g.`, `fig_all_loss_curves.png`, per-optimizer plots).
- ☐ Reviewed all mathematical derivations for correctness and notation clarity.
- ☐ Submitted the completed activity sheet where the answers were written in your `.ipynb` notebook file.

Grading Rubric

Item	Points	Criteria
Q1 — Backpropagation Mechanics	15	Correct forward and backward equations; explanation of computational efficiency.
Q2 — Loss Landscape	10	Specific optimizer names and approximate epoch/loss values; correct interpretation.
Q3 — Adaptive vs. Non-Adaptive	10	Completed comparison table; discussion linked to actual results.
Q4 — Adam Family	15	Correct update rules; numerical comparison; clear AdamW vs. L2 explanation.
Q5 — Vanishing Gradients	10	Correct mathematical derivation; He initialization formula; sigmoid analysis.
Q6 — Adafactor	10	Correct factored approximation description; empirical comparison; use-case rationale.
Q7 — Lion	10	Correct update rule; LR rationale; justified performance prediction.
Q8 — Overfitting	10	Overfitting identified with values; three valid regularization techniques.
Q9 — Muon	10	Correct routing explanation; empirical comparison; deployment recommendation.
Q10 — Design Recommendation	10	Specific optimizer with three justified criteria; reasonable hyperparameter ranges.
Bonus — Convergence	*+10*	Correct normalization interpretation; optimizer identified with reasoning.
Total	**100 + 10**	

Appendix A: Supplementary Notes

Reading the extra results

- **S1 (Muon without exclude_layers):** RMSE 11,157.84, R^2 0.9532, 21 epochs, compared with 11,223.06 / 0.9526 / 22 epochs for the baseline Muon. The difference is small.
- **S2 (Adam with He-normal init):** RMSE 11,234.41, compared with 11,163.13 for the baseline Adam with the Glorot default. Initialization has no meaningful effect in a network with only two hidden layers.
- **S3 (food-group columns removed, 37 features):** Adam RMSE 22,967.61 (R^2 0.8016) and SGD RMSE 22,996.03 (R^2 0.8011), about **double** the baseline RMSE. So most of the baseline R^2 of about 0.95 comes from columns that are parts of the target. The baseline stays as the sheet says, and every R^2 in this work has to be read with that in mind.

Differences from the sheet and limits

- The sheet's Total Income is Total Household Income in this csv (checked in code).
- The sheet's Q5(b) says Keras uses He initialization by default. Keras actually uses Glorot uniform. To get He initialization you have to set kernel_initializer='he_normal'.
- The Muon baseline uses exclude_layers=['output'], as explained in Q9. The version without it is S1.
- Each optimizer was run only once with seed 42, so the comparisons are for this seed only.
- Results come from TensorFlow 2.22.0-rc0 and Keras 3.16.0.dev on CPU. Another machine or library version can give slightly different numbers.

Appendix B: Per-Optimizer Diagnostic Figures

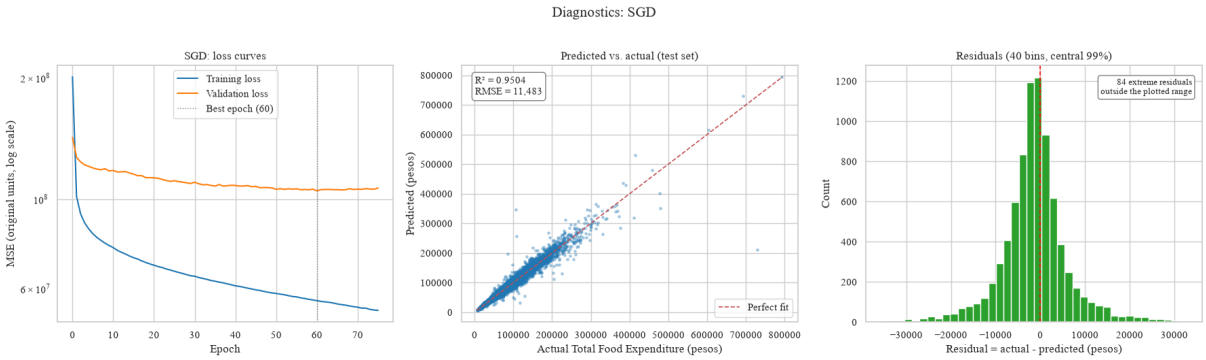


Figure 3: SGD: training/validation loss, predicted vs. actual, and residual histogram.

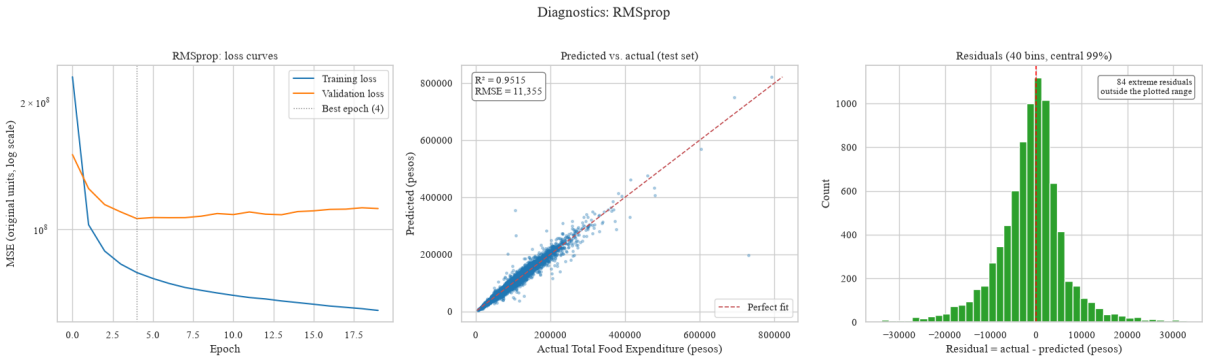


Figure 4: RMSprop: training/validation loss, predicted vs. actual, and residual histogram.

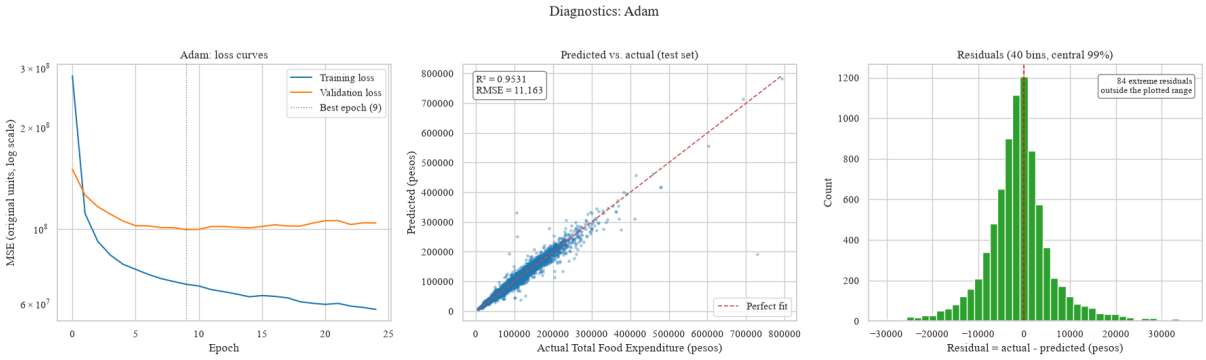


Figure 5: Adam: training/validation loss, predicted vs. actual, and residual histogram.

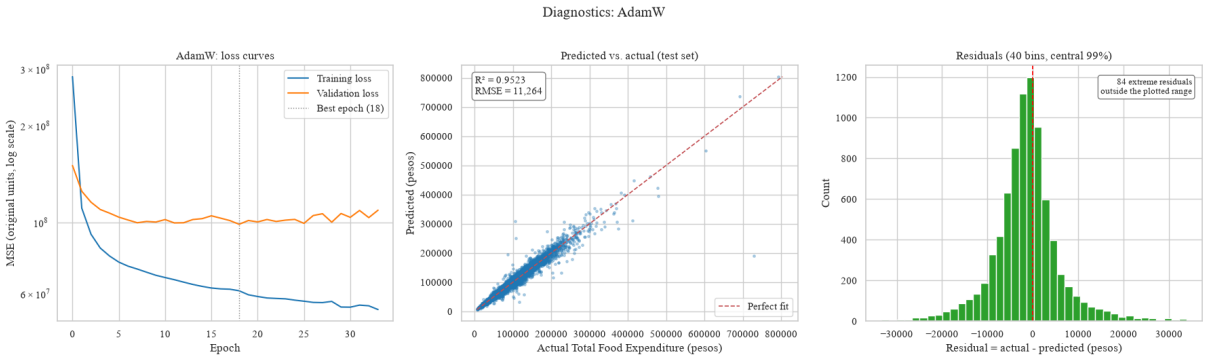


Figure 6: AdamW: training/validation loss, predicted vs. actual, and residual histogram.

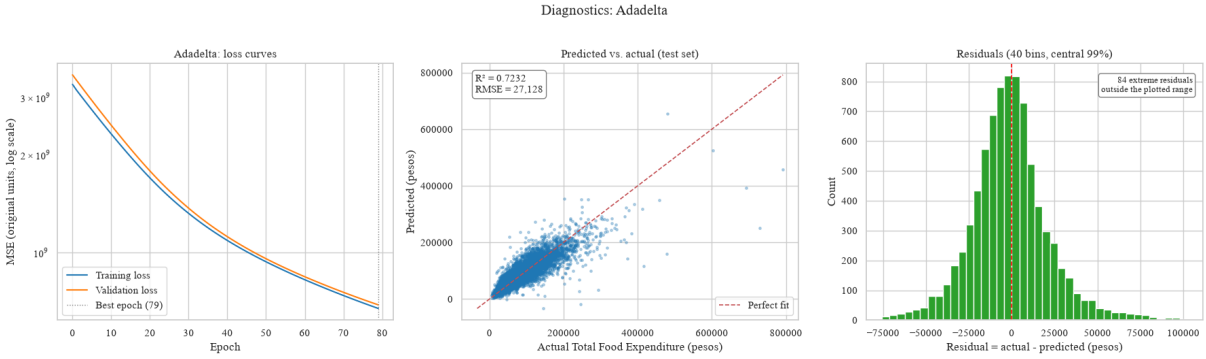


Figure 7: Adadelta: training/validation loss, predicted vs. actual, and residual histogram.

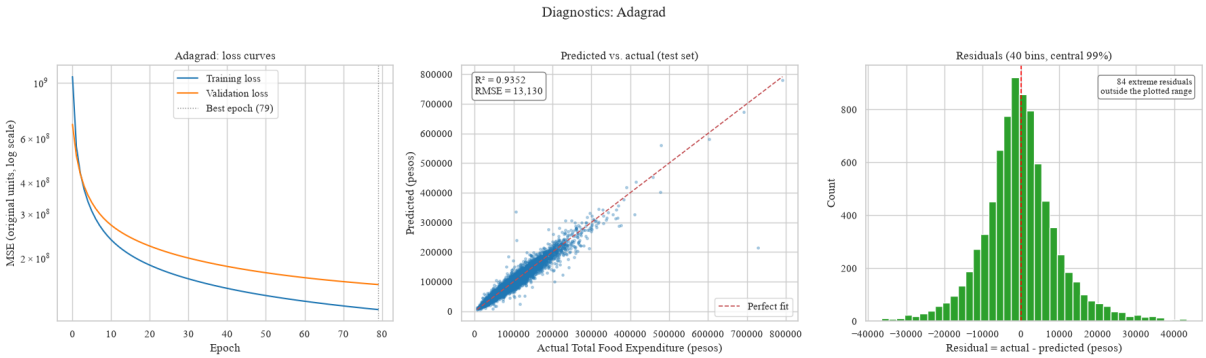


Figure 8: Adagrad: training/validation loss, predicted vs. actual, and residual histogram.

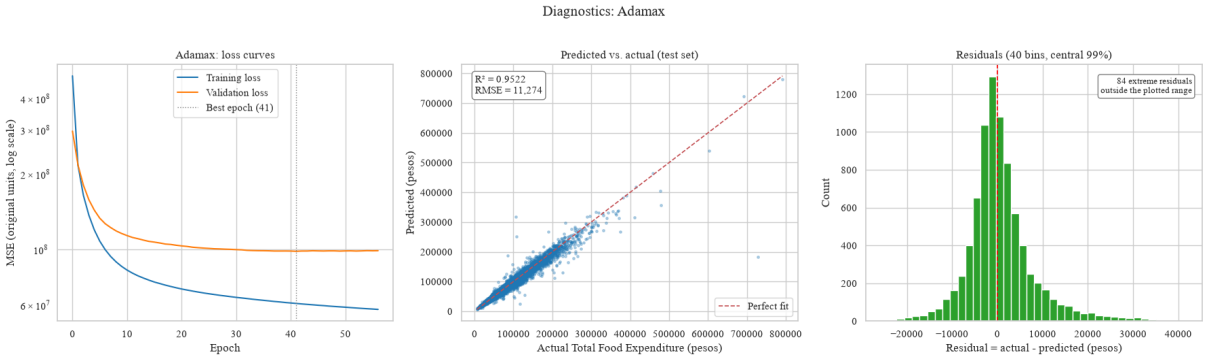


Figure 9: Adamax: training/validation loss, predicted vs. actual, and residual histogram.

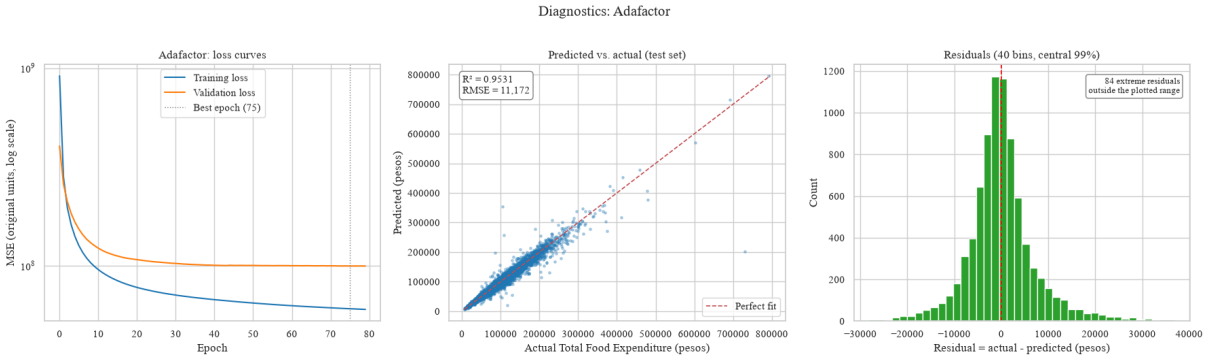


Figure 10: Adafactor: training/validation loss, predicted vs. actual, and residual histogram.

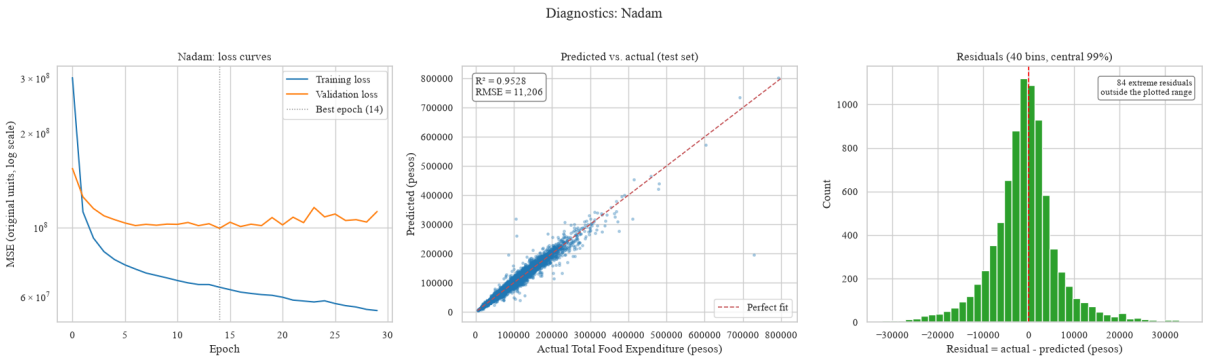


Figure 11: Nadam: training/validation loss, predicted vs. actual, and residual histogram.

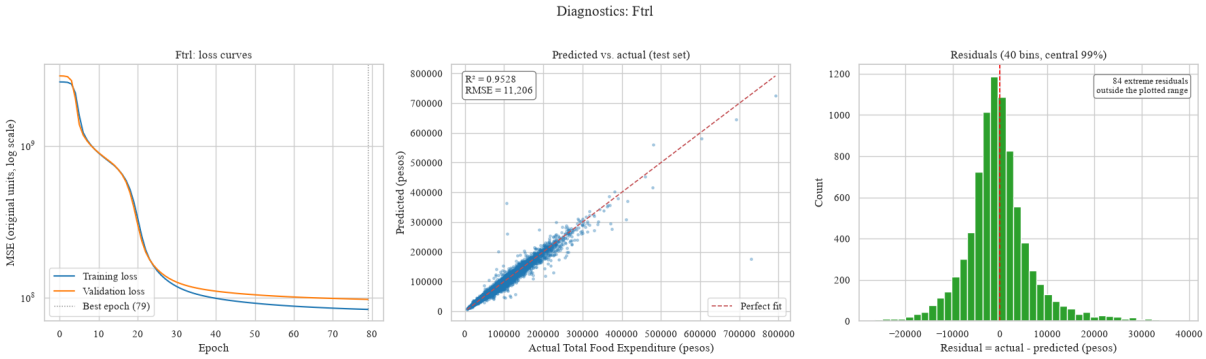


Figure 12: Ftrl: training/validation loss, predicted vs. actual, and residual histogram.

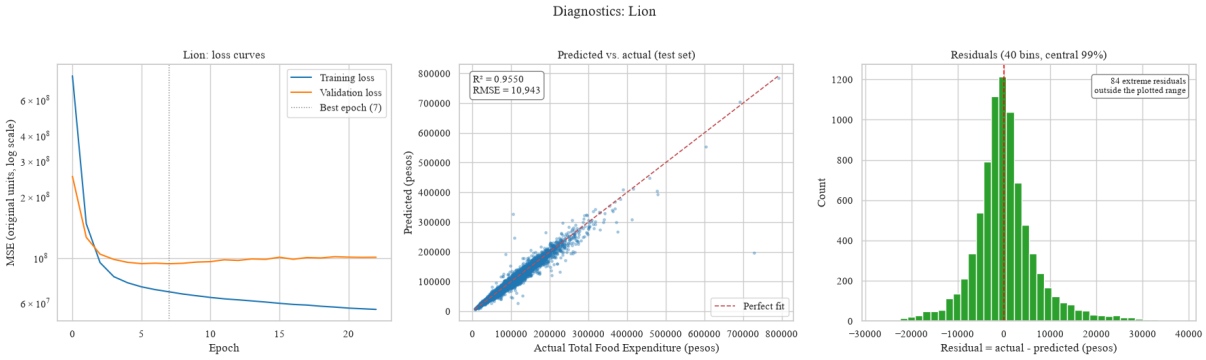


Figure 13: Lion: training/validation loss, predicted vs. actual, and residual histogram.

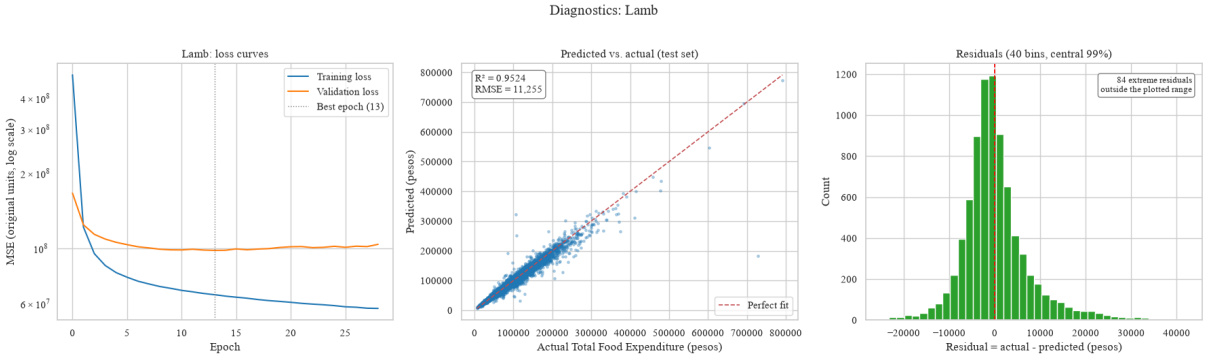


Figure 14: Lamb: training/validation loss, predicted vs. actual, and residual histogram.

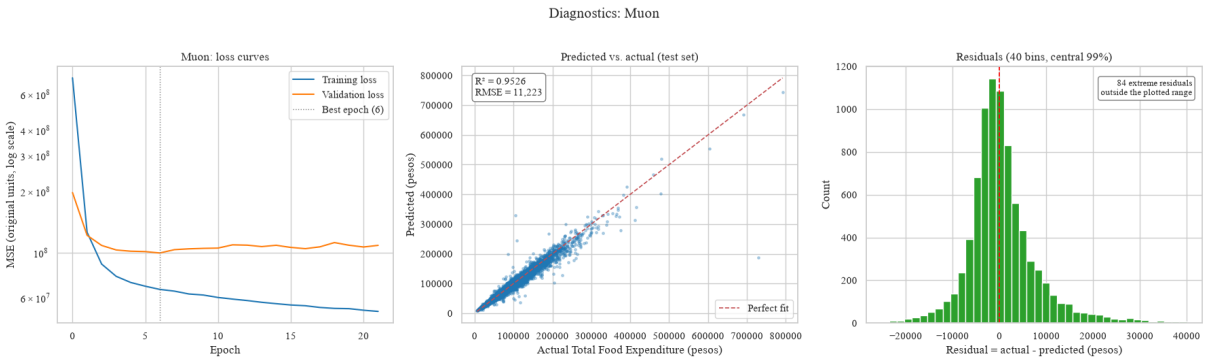


Figure 15: Muon: training/validation loss, predicted vs. actual, and residual histogram.